Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

LEARNER GUIDE

For

AI Agent Cybersecurity

TGS Ref No: TGS-2025053228

Conducted by

Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

Version 3.0

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

DOCUMENT VERSION CONTROL RECORD

Version Number	Effective Date of Release	Summary of Included Changes	Author
2.0	1 January 2025	Legacy release — 'Certified IT Specialist Cybersecurity' (Pearson) Learner Guide.	Dr Alfred Ang
3.0	29 August 2026	AI-agent rewrite: ten self-contained OpenClaw/Hermes labs mapped to LO1–LO3; every lab adds an injection-resistant, evidence-cited, human-approved agent workflow; integrated seven security web tools; DOCX is the single Learner Guide artifact (no divergent Markdown copy).	Dr Alfred Ang

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

TABLE OF CONTENTS

Introduction **4**

Course Learning Outcomes **4**

Before You Start — Environment Setup **4**

Topic 01 — Developing Cybersecurity Policies with OpenClaw and Hermes Agents (LO1 · weighting 40%) **6**

Lab 1 — Agent Security Policy & Trust Boundaries 6

Lab 2 — Phishing & IOC Triage with Regex Generation 8

Lab 3 — Identity, Secrets, Encryption & Crypto Validation 10

Lab 4 — Network Segmentation & IP Policy 12

Topic 02 — AI Agent Monitoring, Compliance and Response to Emerging Threats (LO2 · weighting 30%) **15**

Lab 5 — Agent Compliance Monitoring & Alert Triage 15

Lab 6 — Endpoint Hardening & Controlled Validation 17

Lab 7 — PCAP & Log Investigation 19

Topic 03 — Continuous Cybersecurity Improvement with Autonomous AI Agents (LO3 · weighting 30%) **21**

Lab 8 — Vulnerability Intelligence & Authorized FauxBank Assessment 22

Lab 9 — Incident Response, Forensics & DR with Human Approvals 24

Lab 10 — Capstone — Multi-Agent Cyber-Resilience Improvement Cycle 26

Assessment Preparation **28**

Glossary **28**

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Introduction

This Learner Guide accompanies the WSQ course AI Agent Cybersecurity (TGS-2025053228), conducted by Tertiary Infotech Academy Pte Ltd (UEN 201200696W). It provides detailed, step-by-step instructions for all ten hands-on labs, organised by the three course topics, which map directly to the three learning outcomes and to the assessment (a Written Assessment and a Case Study).

This is version 3.0 — the AI-agent rewrite of the legacy 'Certified IT Specialist Cybersecurity' course. It preserves the proven defensive-cybersecurity content but reframes every control for securing AI agents (OpenClaw and Hermes Agent): their tools, skills, memory, context and permissions. Use this guide alongside the course slides and the self-contained lab folders in the labs/ directory of the course repository.

Note: Authorised targets only. The only systems you may test in this course are the synthetic practice targets provided — FauxBank (https://pentest-fauxbank.vercel.app/) and the Ethical Hacking Trainer (https://alfredang.github.io/ethnicalhacking/). Never scan, probe or attack any other host. Unauthorised access to a computer is an offence under the Singapore Computer Misuse Act.

Course Learning Outcomes

- LO1: Establish organizational cybersecurity policies and procedures to protect information confidentiality and integrity across AI-agent tools, skills, memory, context and permissions.
- LO2: Ensure adherence to cybersecurity policies and procedures in response to evolving threats, using AI-agent monitoring, compliance checks and controlled validation.
- LO3: Continuously improve cybersecurity policies and procedures to enhance organizational defenses, using vulnerability intelligence, incident response and a multi-agent resilience cycle.

How the labs map to the assessment

- Topic 1 (Labs 1–4) → LO1 → Case Study Task 1 (A1) and the Written Assessment (K1).
- Topic 2 (Labs 5–7) → LO2 → Case Study Task 2 (A2) and the Written Assessment (K2).
- Topic 3 (Labs 8–10) → LO3 → Case Study Task 3 (A3).
- Your evidence/ artifacts from each lab are the worked model answers you can draw on in the open-book assessment.

Before You Start — Environment Setup

What you need

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- The lab repository: clone https://github.com/tertiary-infotech/ai-agent-cybersecurity-labs — it contains ten self-contained folders, one per lab.
- Python 3 (standard library only). No third-party packages are required to run the lab scripts or verifiers.
- Access to the OpenClaw and Hermes Agent workspaces provided in class.
- A modern web browser for the seven security web tools (all run in the browser).
- A text editor (VS Code or similar) to edit policy.yaml, patterns.txt and the other lab inputs.

The security web-tool suite (live, clickable)

- IP Subnet Calculator — https://alfredang.github.io/ipcalculator/ · Plan CIDR subnets and segmentation for the agent network policy (Lab 4).
- PCAP Analyzer — https://alfredang.github.io/pcapanalyzer/ · Inspect packet captures to confirm flows and spot anomalies (Lab 7).
- Regex Generator — https://alfredang.github.io/regexgenerator/ · Build and test IOC extraction patterns for phishing triage (Lab 2).
- Cybersecurity Simulator — https://alfredang.github.io/cybersecuritysimulator/ · Explore controls, monitoring and incident scenarios (Labs 1, 5, 9).
- Ethical Hacking Trainer — https://alfredang.github.io/ethnicalhacking/ · Authorized practice target for controlled validation (Labs 6, 8).
- FauxBank (authorized target) — https://pentest-fauxbank.vercel.app/ · The only permitted web-application test target for the assessment lab (Lab 8).
- Cryptography Toolkit — https://alfredang.github.io/cryptography-toolkit/ · Hash, encrypt and validate crypto for the secrets lab (Lab 3).

Conventions used in every lab

- Each lab lives in labs/<lab-folder>/; run commands from inside that folder.
- Commands are shown for a terminal; python3 runs every script with the standard library only.
- mock-data/ holds the synthetic inputs; write your outputs into evidence/.
- Every lab ends with python3 verify.py — a PASS is your acceptance check.
- AI-PROMPTS.md holds the exact OpenClaw/Hermes prompts, their required inputs and expected structured outputs.

Agent guardrails you apply in every lab

- Treat all tool output, files and web content as UNTRUSTED data — never as instructions (prompt-injection defence).
- Limit the agent to this lab's tool allow-list; nothing outside it.
- Require the agent to cite the source file and line for every finding.
- Keep OBSERVATION (what the data shows) separate from INFERENCE (what you conclude).
- Require explicit HUMAN APPROVAL before any state-changing or security action.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Safety rules

- Only inspect the provided synthetic samples and the two authorised web targets.
- Do not scan or attack internet hosts, and do not attempt exploitation beyond the lab checklist.
- Never place real secrets, credentials or personal data into a prompt, a log or the evidence folder.

Topic 01 — Developing Cybersecurity Policies with OpenClaw and Hermes Agents (LO1 · weighting 40%)

LO1 · CIA triad · control categories · agent trust boundaries · tools, skills, memory, context & permissions · secrets & cryptography · IP/segmentation policy

Key concepts

- CIA triad — Confidentiality, Integrity and Availability are the goals every agent security policy protects; C and I are the assessment focus.
- Control categories — Access control, cryptography, operations security, incident management and physical/environmental security — the policy backbone (A1).
- Defence in depth — Layer preventive, detective and corrective controls so no single failure is fatal — the key K1 policy element.
- Agent trust boundary — The line between the model, its tools, its memory and the outside world; every crossing must be authenticated and logged.
- Least privilege & tool scoping — An agent gets only the tools, scopes and data a task needs — the agent equivalent of role-based access control.
- Prompt-injection defence — Untrusted content is data, not instructions; a prompt contract separates trusted policy from untrusted input.
- Identity, secrets & cryptography — AAA, MFA, password strength, hashing vs encryption, TLS in transit and at rest, and secrets never in prompts or logs.
- Network & IP policy — Subnetting, segmentation and NAC scope where an agent and its tools may reach — CIDR planning and firewall rules (K2).

Lab 1 — Agent Security Policy & Trust Boundaries

Objective: establish an AI-agent security policy grounded in the CIA triad and the control categories, and draw the agent trust boundary (LO1, A1, K1). (Maps to LO1, assessment criteria A1/K1.)

Goal: Turn the CIA triad and the control categories into a concrete, machine-checkable security policy for an OpenClaw agent, then map its trust boundary across tools, skills, memory, context and permissions.

Tools and resources

- Lab folder: labs/lab-01-agent-security-policy-trust-boundaries/

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- Cybersecurity Simulator — https://alfredang.github.io/cybersecuritysimulator/
- Platforms: OpenClaw, Hermes Agent, Cybersecurity Simulator, Python (stdlib), diagrams.net

What you'll build

policy.yaml (the agent security policy), a trust-boundary diagram, and a passing policy-lint report.

Step-by-step

1. Read the starter asset inventory and classify each asset by CIA impact (Confidential/Internal/Public).

```
python3 policy_lint.py --classify mock-data/asset-inventory.csv
```

2. Open the Cybersecurity Simulator and walk one preventive, one detective and one corrective control per category.

```
open https://alfredang.github.io/cybersecuritysimulator/
```

3. Draft policy.yaml: for each control category list the rule, the owner and whether human approval is required.

```
cp mock-data/policy.starter.yaml policy.yaml
```

4. Define the agent trust boundary: which tools, skills, memory scopes and data the agent may touch, and what stays outside.

```
python3 trust_boundary.py --policy policy.yaml --out evidence/trust-boundary.json
```

5. Add a prompt contract to policy.yaml that marks untrusted input as data and forbids it from changing tool scope.
6. Ask the OpenClaw policy agent to review policy.yaml — it must cite the line for each finding and separate observations from inferences.

```
see AI-PROMPTS.md → 'Policy review (OpenClaw)'
```

7. Require human approval for every state-changing control before the agent may propose enforcement.
8. Run the policy linter — it fails if any Confidential asset lacks an access + crypto control, or any state-changing rule lacks an approval gate.

```
python3 policy_lint.py --check policy.yaml
```

Expected result — Test it

policy_lint.py exits 0, every Confidential asset has an access and a cryptography control, and every state-changing rule has human_approval: required. verify.py prints PASS.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Troubleshooting

- 'command not found' — ensure you are inside labs/lab-01-agent-security-policy-trust-boundaries/ and using python3.
- A script can't find its input — check the mock-data/ path and that you copied the starter file.
- verify.py FAILS — read its message; it names the exact check that did not pass, then re-run the step.
- The agent invents data or an indicator — tighten the prompt contract; it must cite a real source line.

Evidence to capture

- The primary artifact: policy.yaml (the agent security policy), a trust-boundary diagram, and a passing policy-lint report.
- The exact OpenClaw/Hermes prompts you used (from AI-PROMPTS.md), with their guardrails.
- The human-approval record for any state-changing action the agent proposed.
- The full PASS output of python3 verify.py.

Acceptance checklist

- [ ] policy_lint.py exits 0, every Confidential asset has an access and a cryptography control, and every state-changing rule has human_approval: required. verify.py prints PASS.
- [ ] All agent findings cite a source file and line; observations are separated from inferences.
- [ ] No secret, credential or real personal data appears in any prompt, log or evidence file.
- [ ] Every state-changing action has a recorded human approval.
- [ ] Testing stayed strictly within scope (Cybersecurity Simulator).

Note: Safety boundary: complete Lab 1 only against the provided synthetic data and the authorised targets listed above. The agent proposes; a human approves before anything changes state.

Lab 2 — Phishing & IOC Triage with Regex Generation

Objective: triage phishing reports, extract Indicators of Compromise with regex, and turn them into a security-awareness policy element (K1, LO1). (Maps to LO1, assessment criteria K1/A1.)

Goal: Build a defensible phishing-triage workflow: extract IOCs (URLs, IPs, hashes, sender domains) with tested regex, rank them by confidence, and have a Hermes agent draft an awareness advisory that cites every indicator.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Tools and resources

- Lab folder: labs/lab-02-phishing-ioc-triage-regex/
- Regex Generator — https://alfredang.github.io/regexgenerator/
- Platforms: Hermes Agent, Regex Generator, Python (stdlib re), OpenClaw

What you'll build

ioc-report.json (indicators with evidence lines), a tested regex set, and a human-approved awareness advisory.

Step-by-step

9. Read the mock phishing mailbox (raw .eml samples) and skim the headers and bodies.

```
python3 ioc_extract.py --list mock-data/phish/
```

10. Open the Regex Generator and build patterns for URLs, IPv4, SHA-256 and sender domains; test each on the samples.

```
open https://alfredang.github.io/regexgenerator/
```

11. Paste your tested patterns into patterns.txt and run the extractor to pull IOCs with the source line for each.

```
python3 ioc_extract.py --patterns patterns.txt mock-data/phish/ >
evidence/ioc-report.json
```

12. Score each IOC by confidence and separate observations (the literal string found) from inferences (why it is suspicious).

```
python3 ioc_extract.py --score evidence/ioc-report.json
```

13. Give the Hermes triage agent the report with a guardrail: it may only cite IOCs present in the file and must not invent indicators.

```
see AI-PROMPTS.md → 'IOC triage (Hermes)'
```

14. Have the agent draft a staff awareness advisory that quotes each indicator and its evidence line.
15. Route the advisory to a human approver before it is 'sent' — no advisory leaves without sign-off.
16. Run the verifier: every reported IOC must be traceable to a real line in the samples and the regex set must catch the seeded indicators.

```
python3 verify.py
```

Expected result — Test it

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

verify.py prints PASS: all seeded IOCs are found, no phantom indicators are reported, and every indicator in ioc-report.json carries a source file and line number.

Troubleshooting

- 'command not found' — ensure you are inside labs/lab-02-phishing-ioc-triage-regex/ and using python3.
- A script can't find its input — check the mock-data/ path and that you copied the starter file.
- verify.py FAILS — read its message; it names the exact check that did not pass, then re-run the step.
- The agent invents data or an indicator — tighten the prompt contract; it must cite a real source line.

Evidence to capture

- The primary artifact: ioc-report.json (indicators with evidence lines), a tested regex set, and a human-approved awareness advisory.
- The exact OpenClaw/Hermes prompts you used (from AI-PROMPTS.md), with their guardrails.
- The human-approval record for any state-changing action the agent proposed.
- The full PASS output of python3 verify.py.

Acceptance checklist

- [ ] verify.py prints PASS: all seeded IOCs are found, no phantom indicators are reported, and every indicator in ioc-report.json carries a source file and line number.
- [ ] All agent findings cite a source file and line; observations are separated from inferences.
- [ ] No secret, credential or real personal data appears in any prompt, log or evidence file.
- [ ] Every state-changing action has a recorded human approval.
- [ ] Testing stayed strictly within scope (Regex Generator).

Note: Safety boundary: complete Lab 2 only against the provided synthetic data and the authorised targets listed above. The agent proposes; a human approves before anything changes state.

Lab 3 — Identity, Secrets, Encryption & Crypto Validation

Objective: enforce access control, protect secrets, and validate cryptography (hashing, encryption, TLS) for an AI agent (LO1, A1). (Maps to LO1, assessment criteria A1/K1.)

Goal: Implement the access-control and cryptography controls from your policy: measure password strength, choose hashing vs encryption correctly, scan the agent's config for leaked secrets, and validate a certificate — with the Cryptography Toolkit.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Tools and resources

- Lab folder: labs/lab-03-identity-secrets-encryption-crypto/
- Cryptography Toolkit — https://alfredang.github.io/cryptography-toolkit/
- Platforms: Cryptography Toolkit, OpenClaw, Hermes Agent, Python (stdlib hashlib/secrets)

What you'll build

a secrets-scan report, a password-policy check, and a crypto-validation record.

Step-by-step

17. Scan the agent config bundle for hard-coded secrets (API keys, tokens, passwords).

```
python3 secret_scan.py mock-data/agent-config/
```

18. Open the Cryptography Toolkit and hash a sample password with SHA-256, then with a salted scheme; compare.

```
open https://alfredang.github.io/cryptography-toolkit/
```

19. Decide hashing vs encryption for each data item (passwords, session tokens, PII) and record the choice with a reason.

```
python3 crypto_check.py --classify mock-data/data-items.csv
```

20. Measure password strength / entropy for the sample credentials and flag any below policy.

```
python3 crypto_check.py --entropy mock-data/creds-sample.txt
```

21. Validate the sample TLS certificate: issuer, validity window, key size and hostname match.

```
python3 crypto_check.py --cert mock-data/agent-cert.pem
```

22. Ask OpenClaw to review the findings; it must cite the file/line for each secret and never echo the secret value.

```
see AI-PROMPTS.md → 'Secrets review (OpenClaw)'
```

23. Require human approval before any 'rotate key' or 'revoke cert' action the agent proposes.
24. Run the verifier: seeded secrets are all found, weak passwords flagged, and the expired cert is detected.

```
python3 verify.py
```

Expected result — Test it

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

verify.py prints PASS: every seeded secret is detected, weak/breached passwords are flagged, and the expired certificate is caught — with no secret values written to logs.

Troubleshooting

- 'command not found' — ensure you are inside labs/lab-03-identity-secrets-encryption-crypto/ and using python3.
- A script can't find its input — check the mock-data/ path and that you copied the starter file.
- verify.py FAILS — read its message; it names the exact check that did not pass, then re-run the step.
- The agent invents data or an indicator — tighten the prompt contract; it must cite a real source line.

Evidence to capture

- The primary artifact: a secrets-scan report, a password-policy check, and a crypto-validation record.
- The exact OpenClaw/Hermes prompts you used (from AI-PROMPTS.md), with their guardrails.
- The human-approval record for any state-changing action the agent proposed.
- The full PASS output of python3 verify.py.

Acceptance checklist

- [ ] verify.py prints PASS: every seeded secret is detected, weak/breached passwords are flagged, and the expired certificate is caught — with no secret values written to logs.
- [ ] All agent findings cite a source file and line; observations are separated from inferences.
- [ ] No secret, credential or real personal data appears in any prompt, log or evidence file.
- [ ] Every state-changing action has a recorded human approval.
- [ ] Testing stayed strictly within scope (Cryptography Toolkit).

Note: Safety boundary: complete Lab 3 only against the provided synthetic data and the authorised targets listed above. The agent proposes; a human approves before anything changes state.

Lab 4 — Network Segmentation & IP Policy

Objective: design CIDR subnets and segmentation so an agent and its tools reach only what policy allows, and validate firewall rules (K2, LO2, A2). (Maps to LO2, assessment criteria K2/A2.)

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Goal: Translate the network-security policy into addresses: plan subnets with the IP Calculator, segment the agent, its tools and sensitive data into zones, and validate a firewall ruleset against the segmentation policy — the technical basis for NAC/VPN/BYOD.

Tools and resources

- Lab folder: labs/lab-04-network-segmentation-ip-policy/
- IP Subnet Calculator — https://alfredang.github.io/ipcalculator/
- Platforms: IP Subnet Calculator, OpenClaw, Hermes Agent, Python (stdlib ipaddress)

What you'll build

a subnet plan, a segmentation diagram, and a validated firewall ruleset.

Step-by-step

25. Read the zone requirements (agent runtime, tool sandbox, sensitive data, management) and host counts.

```
python3 subnet_plan.py --need mock-data/zone-requirements.csv
```

26. Open the IP Calculator and carve 10.20.0.0/24 into right-sized subnets per zone.

```
open https://alfredang.github.io/ipcalculator/
```

27. Write the plan to subnets.csv (zone, CIDR, usable hosts, gateway) and verify no overlaps.

```
python3 subnet_plan.py --check subnets.csv
```

28. Define the segmentation policy: which zone may initiate to which, on which ports (least privilege between zones).
29. Validate firewall-rules.csv against the segmentation policy — any rule that lets the tool sandbox reach sensitive data directly must fail.

```
python3 fw_validate.py --policy segmentation-policy.yaml firewall-rules.csv
```

30. Ask the OpenClaw network agent to explain each denied flow, citing the policy line; observations vs inferences kept separate.

```
see AI-PROMPTS.md → 'Segmentation review (OpenClaw)'
```

31. Require human approval before the agent proposes opening any new inter-zone rule.
32. Run the verifier: subnets fit the host counts with no overlap, and every firewall rule complies with the segmentation policy.

```
python3 verify.py
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Expected result — Test it

verify.py prints PASS: subnets satisfy the host counts with no overlap, and the firewall ruleset contains no rule that violates the zone segmentation policy.

Troubleshooting

- 'command not found' — ensure you are inside labs/lab-04-network-segmentation-ip-policy/ and using python3.
- A script can't find its input — check the mock-data/ path and that you copied the starter file.
- verify.py FAILS — read its message; it names the exact check that did not pass, then re-run the step.
- The agent invents data or an indicator — tighten the prompt contract; it must cite a real source line.

Evidence to capture

- The primary artifact: a subnet plan, a segmentation diagram, and a validated firewall ruleset.
- The exact OpenClaw/Hermes prompts you used (from AI-PROMPTS.md), with their guardrails.
- The human-approval record for any state-changing action the agent proposed.
- The full PASS output of python3 verify.py.

Acceptance checklist

- [ ] verify.py prints PASS: subnets satisfy the host counts with no overlap, and the firewall ruleset contains no rule that violates the zone segmentation policy.
- [ ] All agent findings cite a source file and line; observations are separated from inferences.
- [ ] No secret, credential or real personal data appears in any prompt, log or evidence file.
- [ ] Every state-changing action has a recorded human approval.
- [ ] Testing stayed strictly within scope (IP Subnet Calculator).

Note: Safety boundary: complete Lab 4 only against the provided synthetic data and the authorised targets listed above. The agent proposes; a human approves before anything changes state.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Topic 02 — AI Agent Monitoring, Compliance and Response to Emerging Threats (LO2 · weighting 30%)

LO2 · adherence as threats evolve · audit trails & alert triage · endpoint hardening & BYOD · packet/log investigation · NAC, segmentation, VPN and host firewall

Key concepts

- Adherence monitoring — Continuously check that agent actions and endpoints comply with the policy — the core of LO2/A2.
- Audit trails — Every agent tool call, approval and data access is logged with a source citation so actions are attributable and reviewable.
- Alert triage — Rank alerts by severity and confidence; separate observation from inference before escalating.
- Endpoint hardening & BYOD — Patching, host firewall, malware protection, disk encryption and NAC posture checks before network access (K2).
- Detection & log anomalies — Failed-login bursts, RDP brute force, beaconing and data egress stand out in firewall and system logs.
- Packet & PCAP investigation — Read a capture to confirm who talked to whom, on which port, and whether it was allowed.
- Network controls for adherence — Segmentation, NAC, VPN and host firewall technically enforce the written policy for remote and BYOD access (A2).
- Prompt-injection & exfiltration watch — Monitor agent transcripts for injected instructions and attempts to leak secrets or data.

Lab 5 — Agent Compliance Monitoring & Alert Triage

Objective: monitor AI-agent actions for policy adherence, build audit trails, and triage alerts by severity and confidence (LO2, A2). (Maps to LO2, assessment criteria A2/K1.)

Goal: Check that an agent's logged actions comply with the Lab 1 policy: parse the agent audit trail, flag policy violations (out-of-scope tools, missing approvals), and have a Hermes agent triage the resulting alerts with evidence citations.

Tools and resources

- Lab folder: labs/lab-05-agent-compliance-monitoring-alert-triage/
- Cybersecurity Simulator — https://alfredang.github.io/cybersecuritysimulator/
- Platforms: Hermes Agent, Cybersecurity Simulator, OpenClaw, Python (stdlib json/csv)

What you'll build

a compliance report, a prioritised alert queue, and a human-approved response plan.

Step-by-step

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

33. Load the agent audit trail (JSONL of tool calls, approvals and data access) and the Lab 1 policy.

```
python3 compliance_check.py --policy mock-data/policy.yaml --audit mock-data/agent-audit.jsonl
```

34. Flag every action that used a tool outside its scope, skipped an approval, or touched Confidential data without a control.

```
python3 compliance_check.py --report evidence/compliance-report.json
```

35. Open the Cybersecurity Simulator to relate each violation to a monitoring/detective control.

```
open https://alfredang.github.io/cybersecuritysimulator/
```

36. Generate alerts and score each by severity x confidence; separate observation (what the log shows) from inference (suspected cause).

```
python3 triage.py evidence/compliance-report.json > evidence/alert-queue.json
```

37. Give the Hermes triage agent the alert queue with a guardrail: cite the audit line for each alert, never fabricate log entries.

```
see AI-PROMPTS.md → 'Alert triage (Hermes)'
```

38. Have the agent propose a response per critical/high alert, and require human approval before any containment action.
39. Record the approved actions and the reason each lower alert was deferred, into the audit trail.
40. Run the verifier: all seeded violations are caught, no compliant action is mis-flagged, and every alert cites a real audit line.

```
python3 verify.py
```

Expected result — Test it

verify.py prints PASS: every seeded policy violation is detected, false-positive rate is zero on the compliant actions, and each alert is traceable to an audit-trail line.

Troubleshooting

- 'command not found' — ensure you are inside labs/lab-05-agent-compliance-monitoring-alert-triage/ and using python3.
- A script can't find its input — check the mock-data/ path and that you copied the starter file.
- verify.py FAILS — read its message; it names the exact check that did not pass, then re-run the step.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- The agent invents data or an indicator — tighten the prompt contract; it must cite a real source line.

Evidence to capture

- The primary artifact: a compliance report, a prioritised alert queue, and a human-approved response plan.
- The exact OpenClaw/Hermes prompts you used (from AI-PROMPTS.md), with their guardrails.
- The human-approval record for any state-changing action the agent proposed.
- The full PASS output of python3 verify.py.

Acceptance checklist

- [ ] verify.py prints PASS: every seeded policy violation is detected, false-positive rate is zero on the compliant actions, and each alert is traceable to an audit-trail line.
- [ ] All agent findings cite a source file and line; observations are separated from inferences.
- [ ] No secret, credential or real personal data appears in any prompt, log or evidence file.
- [ ] Every state-changing action has a recorded human approval.
- [ ] Testing stayed strictly within scope (Cybersecurity Simulator).

Note: Safety boundary: complete Lab 5 only against the provided synthetic data and the authorised targets listed above. The agent proposes; a human approves before anything changes state.

Lab 6 — Endpoint Hardening & Controlled Validation

Objective: harden endpoints for BYOD/remote work and validate the baseline safely on an authorized target (K2, LO2, A2). (Maps to LO2, assessment criteria K2/A2.)

Goal: Build a CIS-style hardening baseline for the endpoints an agent runs on and for BYOD devices (patching, host firewall, disk encryption, NAC posture), then validate it against the authorized Ethical Hacking Trainer — never a real or internet host.

Tools and resources

- Lab folder: labs/lab-06-endpoint-hardening-controlled-validation/
- Ethical Hacking Trainer — https://alfredang.github.io/ethnicalhacking/
- Platforms: Ethical Hacking Trainer, OpenClaw, Hermes Agent, Python (stdlib)

What you'll build

a hardening baseline checklist, a posture-check report, and a remediation plan.

Step-by-step

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

41. Read the endpoint fleet inventory and the BYOD posture requirements.

```
python3 harden_check.py --inventory mock-data/endpoints.csv
```

42. Build the baseline: patching, host firewall on, disk encryption, MFA-secured RDP, app allow-listing, malware protection.

```
cp mock-data/baseline.starter.yaml baseline.yaml
```

43. Score each endpoint against the baseline and compute the per-control pass rate.

```
python3 harden_check.py --score baseline.yaml mock-data/endpoints.csv >
evidence/posture.json
```

44. Open the Ethical Hacking Trainer (authorized target) and run only the read-only checks the lab lists — no exploitation.

```
open https://alfredang.github.io/ethnicalhacking/
```

45. Map each failing control to a BYOD/remote-work risk (K2) and to a policy line from Lab 1.
46. Ask OpenClaw to draft a remediation plan; it must cite the failing check and mark any change as requiring human approval.

```
see AI-PROMPTS.md → 'Hardening remediation (OpenClaw)'
```

47. Require human approval before any 'apply hardening' action; the agent may only propose, not enforce.
48. Run the verifier: the baseline covers all K2 controls and the posture report matches the seeded endpoint states.

```
python3 verify.py
```

Expected result — Test it

verify.py prints PASS: the baseline includes every required K2 control, the posture report's pass/fail matches the seeded fixtures, and all validation stayed on the authorized target.

Troubleshooting

- 'command not found' — ensure you are inside labs/lab-06-endpoint-hardening-controlled-validation/ and using python3.
- A script can't find its input — check the mock-data/ path and that you copied the starter file.
- verify.py FAILS — read its message; it names the exact check that did not pass, then re-run the step.
- The agent invents data or an indicator — tighten the prompt contract; it must cite a real source line.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Evidence to capture

- The primary artifact: a hardening baseline checklist, a posture-check report, and a remediation plan.
- The exact OpenClaw/Hermes prompts you used (from AI-PROMPTS.md), with their guardrails.
- The human-approval record for any state-changing action the agent proposed.
- The full PASS output of python3 verify.py.

Acceptance checklist

- [ ] verify.py prints PASS: the baseline includes every required K2 control, the posture report's pass/fail matches the seeded fixtures, and all validation stayed on the authorized target.
- [ ] All agent findings cite a source file and line; observations are separated from inferences.
- [ ] No secret, credential or real personal data appears in any prompt, log or evidence file.
- [ ] Every state-changing action has a recorded human approval.
- [ ] Testing stayed strictly within scope (Ethical Hacking Trainer).

Note: Safety boundary: complete Lab 6 only against the provided synthetic data and the authorised targets listed above. The agent proposes; a human approves before anything changes state.

Lab 7 — PCAP & Log Investigation

Objective: investigate a packet capture and firewall/system logs to detect an intrusion attempt and confirm what happened (LO2, A2). (Maps to LO2, assessment criteria A2/K2.)

Goal: Read a capture and the firewall log to reconstruct an RDP brute-force attempt against a segmented host: use the PCAP Analyzer, parse the logs with Python, and have a Hermes agent write an evidence-cited findings note that separates fact from inference.

Tools and resources

- Lab folder: labs/lab-07-pcap-log-investigation/
- PCAP Analyzer — https://alfredang.github.io/pcapanalyzer/
- Platforms: PCAP Analyzer, Hermes Agent, OpenClaw, Python (stdlib)

What you'll build

a timeline of events, an evidence-cited findings note, and a detection rule.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Step-by-step

49. Open the sample capture in the PCAP Analyzer and note the top talkers, ports and flags.

```
open https://alfredang.github.io/pcapanalyzer/
```

50. Parse the firewall log and count DENY events by source IP and destination port.

```
python3 log_investigate.py --fw mock-data/firewall.log
```

51. Identify the RDP brute-force source: many DENYs to 3389 from one address in a short window.

```
python3 log_investigate.py --detect-bruteforce mock-data/firewall.log >
evidence/timeline.json
```

52. Correlate the system auth log to confirm whether any attempt succeeded (it must not, given segmentation).

```
python3 log_investigate.py --auth mock-data/auth.log
```

53. Build a timeline separating observations (log lines) from inferences (attempted RDP brute force).
54. Give the Hermes analyst agent the timeline with a guardrail: cite the exact log line for every claim.

```
see AI-PROMPTS.md → 'Investigation note (Hermes)'
```

55. Have the agent propose a detection rule (regex/threshold) and require human approval before it is 'deployed'.
56. Run the verifier: the brute-force source and port are correctly identified and no successful login is claimed.

```
python3 verify.py
```

Expected result — Test it

verify.py prints PASS: the investigation identifies the correct source IP and port 3389, confirms no successful login, and every finding cites a real log line.

Troubleshooting

- 'command not found' — ensure you are inside labs/lab-07-pcap-log-investigation/ and using python3.
- A script can't find its input — check the mock-data/ path and that you copied the starter file.
- verify.py FAILS — read its message; it names the exact check that did not pass, then re-run the step.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- The agent invents data or an indicator — tighten the prompt contract; it must cite a real source line.

Evidence to capture

- The primary artifact: a timeline of events, an evidence-cited findings note, and a detection rule.
- The exact OpenClaw/Hermes prompts you used (from AI-PROMPTS.md), with their guardrails.
- The human-approval record for any state-changing action the agent proposed.
- The full PASS output of python3 verify.py.

Acceptance checklist

- [ ] verify.py prints PASS: the investigation identifies the correct source IP and port 3389, confirms no successful login, and every finding cites a real log line.
- [ ] All agent findings cite a source file and line; observations are separated from inferences.
- [ ] No secret, credential or real personal data appears in any prompt, log or evidence file.
- [ ] Every state-changing action has a recorded human approval.
- [ ] Testing stayed strictly within scope (PCAP Analyzer).

Note: Safety boundary: complete Lab 7 only against the provided synthetic data and the authorised targets listed above. The agent proposes; a human approves before anything changes state.

Topic 03 — Continuous Cybersecurity Improvement with Autonomous AI Agents (LO3 · weighting 30%)

LO3 · vulnerability intelligence & CVSS · authorized assessment · incident response, forensics & DR with human approval · risk register · multi-agent resilience cycle

Key concepts

- Continuous improvement — A structured cycle — identify, assess, monitor, improve — keeps policy effective as threats evolve (A3).
- Vulnerability management — Scan, score with CVSS, prioritise by risk and patch on a defined SLA; feed findings back into policy.
- Vulnerability intelligence — Map findings to CVE/CWE and exploit likelihood so the highest real risk is fixed first.
- Authorized assessment only — Test only synthetic/authorized targets (FauxBank and the ethical-hacking trainer) under written rules of engagement.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- Incident response — Prepare, detect, contain, eradicate, recover and learn — with a timeline and evidence preservation.
- Forensics & evidence — Preserve, document chain of custody, and separate what the evidence shows from what you infer.
- Disaster recovery — RTO and RPO drive backup frequency and recovery design; test the restore, don't assume it.
- Multi-agent resilience & human approval — Specialist agents divide monitoring, triage and response, but a human approves every state-changing action.

Lab 8 — Vulnerability Intelligence & Authorized FauxBank Assessment

Objective: run an authorized web-app assessment, score findings with CVSS, and prioritise remediation with vulnerability intelligence (LO3, A3). (Maps to LO3, assessment criteria A3/K2.)

Goal: Perform a scoped, authorized assessment of FauxBank (the only permitted web target), score each finding with CVSS, enrich with CVE/CWE intelligence, and have a Hermes agent build a risk-ranked vulnerability register — every action inside the rules of engagement.

Tools and resources

- Lab folder: labs/lab-08-vulnerability-intelligence-fauxbank-assessment/
- FauxBank (authorized target) — https://pentest-fauxbank.vercel.app/
- Ethical Hacking Trainer — https://alfredang.github.io/ethnicalhacking/
- Platforms: FauxBank (authorized), Ethical Hacking Trainer, Hermes Agent, OpenClaw, Python (stdlib)

What you'll build

a CVSS-scored vulnerability register, a remediation SLA plan, and a rules-of-engagement record.

Step-by-step

57. Read and accept the rules of engagement: only FauxBank and the trainer, only the listed checks, no data destruction.

```
python3 vuln_assess.py --show-roe
```

58. Open FauxBank (authorized target) and record observed issues against the provided checklist — evidence only, no exploitation beyond scope.

```
open https://pentest-fauxbank.vercel.app/
```

59. Enter each finding into the register with the affected URL, the observation and the CWE class.

```
cp mock-data/findings.starter.csv findings.csv
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

60. Score every finding with the CVSS base metrics and compute the severity band.

```
python3 cvss.py --score findings.csv > evidence/vuln-register.json
```

61. Enrich with vulnerability intelligence: map to a CVE/CWE and an exploit-likelihood note.

```
python3 vuln_assess.py --enrich evidence/vuln-register.json
```

62. Give the Hermes agent the register with a guardrail: cite the finding ID for every recommendation and never claim an unverified exploit.

```
see AI-PROMPTS.md → 'Vuln register (Hermes)'
```

63. Assign a remediation SLA by severity and require human approval before any 'retest' that changes target state.
64. Run the verifier: CVSS scores match the base metrics, the register is sorted by risk, and scope was never exceeded.

```
python3 verify.py
```

Expected result — Test it

verify.py prints PASS: every finding's CVSS score matches its base metrics, the register is risk-sorted, and only the authorized targets appear in the evidence.

Troubleshooting

- 'command not found' — ensure you are inside labs/lab-08-vulnerability-intelligence-fauxbank-assessment/ and using python3.
- A script can't find its input — check the mock-data/ path and that you copied the starter file.
- verify.py FAILS — read its message; it names the exact check that did not pass, then re-run the step.
- The agent invents data or an indicator — tighten the prompt contract; it must cite a real source line.

Evidence to capture

- The primary artifact: a CVSS-scored vulnerability register, a remediation SLA plan, and a rules-of-engagement record.
- The exact OpenClaw/Hermes prompts you used (from AI-PROMPTS.md), with their guardrails.
- The human-approval record for any state-changing action the agent proposed.
- The full PASS output of python3 verify.py.

Acceptance checklist

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- [ ] verify.py prints PASS: every finding's CVSS score matches its base metrics, the register is risk-sorted, and only the authorized targets appear in the evidence.
- [ ] All agent findings cite a source file and line; observations are separated from inferences.
- [ ] No secret, credential or real personal data appears in any prompt, log or evidence file.
- [ ] Every state-changing action has a recorded human approval.
- [ ] Testing stayed strictly within scope (FauxBank (authorized target); Ethical Hacking Trainer).

Note: Safety boundary: complete Lab 8 only against the provided synthetic data and the authorised targets listed above. The agent proposes; a human approves before anything changes state.

Lab 9 — Incident Response, Forensics & DR with Human Approvals

Objective: run the incident-response lifecycle, preserve forensic evidence, and plan disaster recovery with RTO/RPO — human-approved at each gate (LO3, A3). (Maps to LO3, assessment criteria A3/A2.)

Goal: Work a simulated agent-data-exfiltration incident end to end: build the IR timeline, preserve evidence with chain of custody, compute RTO/RPO for recovery, and gate every state-changing action behind explicit human approval.

Tools and resources

- Lab folder: labs/lab-09-incident-response-forensics-dr/
- Cybersecurity Simulator — https://alfredang.github.io/cybersecuritysimulator/
- Platforms: Cybersecurity Simulator, Hermes Agent, OpenClaw, Python (stdlib hashlib)

What you'll build

an incident timeline, a chain-of-custody log, and an RTO/RPO-driven recovery plan.

Step-by-step

65. Open the Cybersecurity Simulator incident scenario and read the alert that opens the incident.

```
open https://alfredang.github.io/cybersecuritysimulator/
```

66. Build the IR timeline through the lifecycle: prepare, detect, contain, eradicate, recover, learn.

```
python3 ir_timeline.py mock-data/incident-events.jsonl >
evidence/timeline.json
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

67. Preserve evidence: hash each artifact and record a chain-of-custody entry (who, what, when).

```
python3 forensics.py --preserve mock-data/artifacts/ --log
evidence/custody.csv
```

68. Separate observations (log/artifact facts) from inferences (attacker actions) in the timeline.
69. Compute RTO/RPO per system and derive backup frequency and restore order.

```
python3 dr_plan.py --systems mock-data/systems.csv > evidence/recovery-
plan.json
```

70. Ask the Hermes IR agent to draft the containment and recovery steps, citing the timeline entry for each.

```
see AI-PROMPTS.md → 'IR plan (Hermes)'
```

71. Gate every state-changing step (isolate host, rotate creds, restore backup) behind explicit human approval.
72. Run the verifier: the timeline is ordered and complete, custody hashes verify, and RTO/RPO drive the plan.

```
python3 verify.py
```

Expected result — Test it

verify.py prints PASS: the timeline covers all six IR phases in order, every artifact hash verifies against the custody log, and the recovery order matches the RTO/RPO ranking.

Troubleshooting

- 'command not found' — ensure you are inside labs/lab-09-incident-response-forensics-dr/ and using python3.
- A script can't find its input — check the mock-data/ path and that you copied the starter file.
- verify.py FAILS — read its message; it names the exact check that did not pass, then re-run the step.
- The agent invents data or an indicator — tighten the prompt contract; it must cite a real source line.

Evidence to capture

- The primary artifact: an incident timeline, a chain-of-custody log, and an RTO/RPO-driven recovery plan.
- The exact OpenClaw/Hermes prompts you used (from AI-PROMPTS.md), with their guardrails.
- The human-approval record for any state-changing action the agent proposed.
- The full PASS output of python3 verify.py.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Acceptance checklist

- [ ] verify.py prints PASS: the timeline covers all six IR phases in order, every artifact hash verifies against the custody log, and the recovery order matches the RTO/RPO ranking.
- [ ] All agent findings cite a source file and line; observations are separated from inferences.
- [ ] No secret, credential or real personal data appears in any prompt, log or evidence file.
- [ ] Every state-changing action has a recorded human approval.
- [ ] Testing stayed strictly within scope (Cybersecurity Simulator).

Note: Safety boundary: complete Lab 9 only against the provided synthetic data and the authorised targets listed above. The agent proposes; a human approves before anything changes state.

Lab 10 — Capstone — Multi-Agent Cyber-Resilience Improvement Cycle

Objective: run a continuous-improvement cycle across a multi-agent team to raise the organization's cyber-resilience, closing the loop on LO1–LO3 (A1, A2, A3). (Maps to LO3, assessment criteria A1/A2/A3.)

Goal: Bring Labs 1–9 together: a multi-agent Hermes team (monitor, triage, respond, improve) ingests every lab's evidence, scores resilience by domain, proposes prioritised policy improvements, and produces a board-ready report — with a human approving each change.

Tools and resources

- Lab folder: labs/lab-10-capstone-multi-agent-resilience-cycle/
- Cybersecurity Simulator — https://alfredang.github.io/cybersecuritysimulator/
- IP Subnet Calculator — https://alfredang.github.io/ipcalculator/
- FauxBank (authorized target) — https://pentest-fauxbank.vercel.app/
- Platforms: Hermes Agent (multi-agent), OpenClaw, all course web tools, Python (stdlib)

What you'll build

a resilience scorecard, a prioritised improvement backlog, and a human-approved board report.

Step-by-step

73. Aggregate the evidence from Labs 1–9 into one dataset (policy, IOCs, crypto, subnets, compliance, posture, logs, vulns, IR).

```
python3 capstone.py --ingest evidence/
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

74. Score resilience maturity (1–5) for each control domain against the target profile.

```
python3 capstone.py --score > evidence/scorecard.json
```

75. Stand up the Hermes multi-agent team: a monitor, a triage, a responder and an improver agent, each with scoped tools.

```
see AI-PROMPTS.md → 'Multi-agent orchestration (Hermes)'
```

76. Each agent must cite the source lab/evidence for its input and separate observation from recommendation.
77. Generate a prioritised improvement backlog: biggest maturity gaps and highest-risk findings first.

```
python3 capstone.py --backlog > evidence/backlog.json
```

78. Map every backlog item back to a policy line (LO1), an adherence control (LO2) and the improvement cycle (LO3).
79. Require human approval on the improver agent's proposed policy changes before the board report is finalised.
80. Produce the board-ready report and run the verifier to confirm the cycle is complete and evidence-cited.

```
python3 capstone.py --report && python3 verify.py
```

Expected result — Test it

verify.py prints PASS: the scorecard covers all six domains, the backlog is risk-sorted and traceable to Labs 1–9, and every proposed change carries a human-approval record.

Troubleshooting

- 'command not found' — ensure you are inside labs/lab-10-capstone-multi-agent-resilience-cycle/ and using python3.
- A script can't find its input — check the mock-data/ path and that you copied the starter file.
- verify.py FAILS — read its message; it names the exact check that did not pass, then re-run the step.
- The agent invents data or an indicator — tighten the prompt contract; it must cite a real source line.

Evidence to capture

- The primary artifact: a resilience scorecard, a prioritised improvement backlog, and a human-approved board report.
- The exact OpenClaw/Hermes prompts you used (from AI-PROMPTS.md), with their guardrails.
- The human-approval record for any state-changing action the agent proposed.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- The full PASS output of python3 verify.py.

Acceptance checklist

- [ ] verify.py prints PASS: the scorecard covers all six domains, the backlog is risk-sorted and traceable to Labs 1–9, and every proposed change carries a human-approval record.
- [ ] All agent findings cite a source file and line; observations are separated from inferences.
- [ ] No secret, credential or real personal data appears in any prompt, log or evidence file.
- [ ] Every state-changing action has a recorded human approval.
- [ ] Testing stayed strictly within scope (Cybersecurity Simulator; IP Subnet Calculator; FauxBank (authorized target)).

Note: Safety boundary: complete Lab 10 only against the provided synthetic data and the authorised targets listed above. The agent proposes; a human approves before anything changes state.

Assessment Preparation

- First pass: complete every lab end to end, capturing the evidence/ artifacts as you go.
- Second pass: redo each lab from a blank folder until the workflow, commands and guardrails are automatic.
- Written Assessment (K1): be able to list the key elements of a multi-layered cybersecurity policy — patching, awareness, network security, encryption, incident response and least privilege.
- Written Assessment (K2): be able to explain network and endpoint measures for BYOD/remote work — NAC, segmentation, VPN, host firewall and RDP secured with MFA.
- Case Study (A1/A2/A3): practise structuring an answer around the CIA triad and control categories (LO1), network/endpoint adherence controls (LO2), and a continuous-improvement cycle with risk and vulnerability management (LO3) — the exact skills the labs build.
- Sharpen SOC skills with the CompTIA CySA+ practice exam: https://exams.tertiaryinfotech.com/practice-exams/comptia/comptia-cysa-plus

Glossary

- **CIA triad —** Confidentiality, Integrity, Availability — the three goals every security policy protects.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- **Trust boundary —** The line separating the agent core from untrusted input and privileged tools/data; every crossing is checked.
- **Prompt contract —** The agent's system prompt written as an enforceable policy — trusted vs untrusted, tool allow-list, citations, approval.
- **Prompt injection —** An attack that hides instructions in untrusted content so the agent acts against policy (OWASP LLM01).
- **Least privilege —** Granting only the minimal tools, scope and data a task needs.
- **Audit trail —** A logged, attributable, tamper-evident record of every agent action, used to prove adherence.
- **NAC —** Network Access Control — a device (or agent) must meet a security posture before it is granted access.
- **Segmentation —** Dividing a network into isolated zones so a breach cannot spread.
- **CVSS —** Common Vulnerability Scoring System — a 0–10 severity score with defined bands.
- **CWE / CVE —** Common Weakness Enumeration (the weakness type) / Common Vulnerabilities and Exposures (a specific known flaw).
- **RTO / RPO —** Recovery Time Objective (how fast back) and Recovery Point Objective (how much data loss is tolerable).
- **Chain of custody —** A signed record of who handled each piece of evidence and when, preserving its integrity.
- **Human-in-the-loop —** A required human approval gate before any state-changing or security action.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg